UNIT II: Functions and Classes & Objects in C++

Question Bank Answers: Debugging & Pseudo-Code Formulation

Name	Nikitha R
Reg. No	19252421
Course Code & Name	DSA0105 Object Oriented Programming with C++

Contents

- **Section A: Debugging** (5 problems): line numbers and defect classification, root cause analysis, corrected code and dry run with the predicted terminal output.
- **Section B: Pseudo-Code** (5 problems): design overview, structured pseudo-code and a trace table.
- Every corrected C++ program was compiled with g++ (`-Wall -Wextra`) and run; the outputs shown are the real outputs. The buggy originals were compiled too, so the quoted compiler messages are real GCC messages.

SECTION A: DEBUGGING QUESTIONS

Question 1: Dangling Reference in Return-by-Reference

Topic: Return by Reference, Scope, Activation Records

Step 1: Analyse the problem statement

- **Expected:** `getMaximumScaled` returns the scaled maximum (40) and `getArrayElement` lets the caller overwrite an array element.
- **Actual:** the array function works, but the scaled-maximum reference points at a variable that no longer exists.

Step 2: Errors found

Line	Classification	Defect
Line 8: `return result;`	Undefined Behavior (dangling reference); GCC warns	Returns a reference to the automatic variable `result`. GCC: "reference to local variable 'result' returned [-Wreturn-local-addr]".
Line 19: `int &danglingRef = getMaximumScaled(...)`	Undefined Behavior	Binds a long-lived reference to a stack slot that was already released.
Line 27: reading `danglingRef`	Runtime Bug	Reads that dead slot after other calls have reused it: garbage value or segmentation fault.

Step 3: Root cause explanation

- **Task 1 (what happens to result).** Every call pushes an activation record (stack frame) holding `a, b, scaleFactor, maxVal, result`. On `return` the frame is popped: the stack pointer moves back, so the slot that held `result` is no longer owned by anything. The bytes may still contain 40 for a moment, but the next call (`std::cout` operator calls, `getArrayElement`) reuses that memory and overwrites it. `danglingRef` is still bound to that address, so reading it on Line 27 is undefined behavior: stale data, garbage, or a crash.
- **Task 2 (why getArrayElement is safe).** The object returned, `arr[index]`, belongs to the *caller*: the array `data` lives in `main()`'s frame, which outlives the call. A function may return a reference only to an object whose lifetime extends beyond the function: caller-owned data, heap data, a static/global variable, or a data member of a still-living object. `result` is an automatic local of the callee, so its lifetime ends at the closing brace.
- **Lvalue rule.** Because `getArrayElement` returns `int&`, the call expression is an lvalue, which is why `getArrayElement(data, 1) = 999;` is legal and writes directly into `data[1]`.

Step 4: Corrected code (return by value for the computed result)

```
#include <iostream>

// Return BY VALUE: the caller receives its own copy, nothing dangles
int getMaximumScaled(int a, int b, int scaleFactor) {
    int maxVal = (a > b) ? a : b;
    return maxVal * scaleFactor;
}

// Safe return-by-reference: arr[index] lives in the caller's array
int& getArrayElement(int arr[], int index) {
    return arr[index];
}

int main() {
    int x = 10, y = 20;

    int scaled = getMaximumScaled(x, y, 2);          // independent copy
    std::cout << "Scaled Max: " << scaled << std::endl;

    int data[3] = {100, 200, 300};
    getArrayElement(data, 1) = 999;                  // lvalue assignment
    std::cout << "Updated Array[1]: " << data[1] << std::endl;

    std::cout << "Scaled Value (still valid): " << scaled << std::endl;
    return 0;
}
```

Step 5: Dry run and predicted output

Step	Action	State
1	getMaximumScaled(10, 20, 2): maxVal = 20; return 20 * 2	scaled = 40 (copied into main's variable)
2	print scaled	Scaled Max: 40
3	getArrayElement(data, 1) returns alias of data[1]; assign 999	data = {100, 999, 300}
4	print data[1]	Updated Array[1]: 999
5	print scaled again: it is an independent int, unaffected by later calls	Scaled Value (still valid): 40

```
Scaled Max: 40
Updated Array[1]: 999
Scaled Value (still valid): 40
```

Question 2: Static Data Member Linkage & Member Function Scoping

Topic: Static Data Members, Static Member Functions, Scope Resolution

Step 1: Analyse the problem statement

- **Expected:** a class-wide reserve total, plus a static audit function that prints it.
- **Actual:** one compile error (static function touching instance data) and one link error (static member never defined).

Step 2: Errors found

Line	Classification	Defect
Line 8: `static double totalBankReserves;`	Linker Error	Only declared, never defined. ld: "undefined reference to 'BankAccount::totalBankReserves'" (reported for the constructor and for displayAudit).
Line 19: `accHolder` inside `displayAudit()`	Compilation Error (semantic)	GCC: "invalid use of member 'BankAccount::accHolder' in static member function".

Step 3: Root cause explanation

- **Task 1 (linker).** Inside the class body, `static double totalBankReserves;` is only a *declaration*: it tells the compiler the name and type exist. A static data member is **not** part of any object, so no constructor can allocate it, and there is no object layout to hold it. Exactly one translation unit must *define* it at namespace (file) scope, `double BankAccount::totalBankReserves = 0.0;`, which reserves one shared storage location and gives the initial value. Without that, the compiler succeeds, but the linker finds references with no definition (the One Definition Rule is not met).
- **Task 2 (compile).** A static member function belongs to the class, not to an object, so it is called as `BankAccount::displayAudit()` and receives **no** `this` **pointer**. Writing `accHolder` means `this->accHolder`, but there is no `this` and no object to read the name from (there are two accounts: whose name?). A static function may use static members and its parameters only. To show a particular holder's data, use a separate *non-static* member function, or pass an object to the static one.
- **Scope resolution.** `::` in `BankAccount::totalBankReserves` tells the compiler that the definition belongs to the class scope.

Step 4: Corrected code

```
#include <iostream>
#include <string>

class BankAccount {
private:
    std::string accHolder;
    double balance;
    static double totalBankReserves;      // declaration only

public:
    BankAccount(std::string name, double initialDeposit)
        : accHolder(name), balance(initialDeposit) {
        totalBankReserves += initialDeposit;
    }

    // Static: touches class-level data only
    static void displayAudit() {
        std::cout << "Total Reserves in Vault: $" << totalBankReserves << std::endl;
    }

    // Non-static: has 'this', so it may use instance data
    void displayAccount() const {
        std::cout << "Account Holder: " << accHolder
                  << ", Balance: $" << balance << std::endl;
    }
};

double BankAccount::totalBankReserves = 0.0;   // definition: allocates storage

int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    acc1.displayAccount();
    acc2.displayAccount();
    BankAccount::displayAudit();
    return 0;
}
```

Step 5: Dry run and predicted output

Statement	totalBankReserves	Notes
start of program (file-scope definition)	0.0	storage exists before main()
acc1("Alice", 5000)	5000	constructor adds the deposit
acc2("Bob", 3000)	8000	one shared variable for both objects
acc1.displayAccount(), acc2.displayAccount()	8000	non-static: uses each object's own accHolder and balance
BankAccount::displayAudit()	8000	called with the class name: no object needed

```
Account Holder: Alice, Balance: $5000
Account Holder: Bob, Balance: $3000
Total Reserves in Vault: $8000
```

Question 3: Function Overload Ambiguity via Default Arguments

Topic: Function Overloading, Default Arguments

Step 1: Analyse the problem statement

- **Expected:** one call for a cube, one for a cylinder, with an optional cuboid form.
- **Actual:** GCC rejects *both* calls: Line 19 (`computeVolume(5.0)`) and Line 22 (`computeVolume(3.0, 7.0)`).

Step 2: Errors found

Line	Classification	Defect
Line 19: `computeVolume(5.0)`	Compilation Error (ambiguous overload)	GCC: "call of overloaded 'computeVolume(double)' is ambiguous". Candidates: Overload 1 (defaults make it callable with 1 argument) and Overload 2 `(double)`.
Line 22: `computeVolume(3.0, 7.0)`	Compilation Error (ambiguous overload)	GCC: "call of overloaded 'computeVolume(double, double)' is ambiguous". Candidates: Overload 1 (one default used) and Overload 3 `(double, double)`.

Step 3: Root cause explanation

- **Task 1.** Overload resolution looks only at the *number and types of parameters actually supplied* by the call, not at what the programmer intended. A default argument does not create a separate function; it makes one function viable for several argument counts. With defaults, Overload 1 is viable for 1, 2 and 3 arguments. For `computeVolume(5.0)` both Overload 1 (`5.0, 1.0, 1.0`) and Overload 2 (`5.0`) accept one `double` with an **exact match** on every supplied argument. Neither is better on any argument (defaults are not considered in ranking), so the call is ambiguous.
- **Task 2.** Yes, Line 22 conflicts as well. `computeVolume(3.0, 7.0)` is viable for Overload 1 (length = 3.0, width = 7.0, height defaults to 1.0) and for Overload 3 (radius = 3.0, height = 7.0). Both are exact matches for both arguments, so there is no unique best viable function. The matching process is: (1) build the candidate set by name, (2) keep the viable ones (count and convertibility), (3) rank by conversion quality per argument, (4) if one function is not strictly better than all others, report ambiguity. The task statement assumed only Line 19 fails; in fact both fail.
- **Fix strategy.** Make every signature distinguishable by parameter count/types without overlapping defaults, or give functions with different meanings different names.

Step 4: Corrected code (refactored signatures)

```
#include <iostream>

// Cube: 1 argument
double computeVolume(double side) {
    return side * side * side;
}

// Cylinder: 2 arguments
double computeVolume(double radius, double height) {
    return 3.14159 * radius * radius * height;
}

// Cuboid: 3 arguments (no default arguments, so no overlap with the others)
double computeVolume(double length, double width, double height) {
    return length * width * height;
}

int main() {
    std::cout << "Cube: "     << computeVolume(5.0) << std::endl;
    std::cout << "Cylinder: " << computeVolume(3.0, 7.0) << std::endl;
    std::cout << "Cuboid: "   << computeVolume(2.0, 3.0, 4.0) << std::endl;
    return 0;
}
```

Alternative: keep a flexible cuboid with defaults under its own name, `cuboidVolume(double l, double w = 1.0, double h = 1.0)`, and leave `computeVolume` for cube/cylinder only. Different names remove any possibility of overlap.

Step 5: Dry run and output

Call	Viable overloads	Chosen	Computation	Result
computeVolume(5.0)	(double) only	cube	5 × 5 × 5	125
computeVolume(3.0, 7.0)	(double, double) only	cylinder	3.14159 × 9 × 7 = 197.92017	197.92 (6 significant digits)
computeVolume(2.0, 3.0, 4.0)	(double, double, double) only	cuboid	2 × 3 × 4	24

```
Cube: 125
Cylinder: 197.92
Cuboid: 24
```

Question 4: Array of Objects Missing Default Constructor

Topic: Arrays of Objects, Constructors, Class Specification

Step 1: Analyse the problem statement

- **Expected:** create three students, fill them, and print them.
- **Actual:** `Student batch[3];` does not compile.

Step 2: Error found

Line	Classification	Defect
Line 20: `Student batch[3];`	Compilation Error	GCC: "no matching function for call to 'Student::Student()'". Declaring an array default-constructs every element, but the class has no zero-argument constructor.

Step 3: Root cause explanation (Tasks 1 and 2)

- **Rule of implicit synthesis.** If a class declares *no* constructor at all, the compiler synthesizes a public default constructor `Student()`. As soon as the programmer declares *any* constructor (here `Student(int, std::string)`), that implicit default constructor is **not** generated.
- An array declaration `Student batch[3];` has to create three objects with no arguments to supply, so it needs an accessible zero-argument constructor. None exists, hence the error. The later assignments `batch[0] = Student(101, "Aarav")` are irrelevant: the compile error occurs at the declaration, before any assignment.
- **Technique A:** add an explicit default constructor `Student() : rollNumber(0), name("Unassigned") {}` (or give every parameter a default value). Elements are created with placeholder data and filled in afterwards.
- **Technique B:** initialize each element through an array initializer list: `Student batch[3] = { Student(101,"Aarav"), Student(102,"Bhavna"), Student(103,"Chirag") };`. The parameterized constructor builds each element directly, so no default constructor is required (compiled and verified, same output).

Step 4: Corrected code using Technique A

```
#include <iostream>
#include <string>

class Student {
private:
    int rollNumber;
    std::string name;

public:
    Student() : rollNumber(0), name("Unassigned") {}      // Technique A: explicit default ctor

    Student(int r, std::string n) : rollNumber(r), name(n) {}

    void display() const {
        std::cout << "Roll: " << rollNumber << ", Name: " << name << std::endl;
    }
};

int main() {
    Student batch[3];                                     // uses Student()

    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");

    for (int i = 0; i < 3; ++i) {
        batch[i].display();
    }
    return 0;
}
```

Step 5: Dry run and verified output

Step	Action	batch state
1	Student batch[3]; calls Student() three times	{0, Unassigned} x 3
2	batch[0] = Student(101, "Aarav")	[0] = {101, Aarav}
3	batch[1] = Student(102, "Bhavna"); batch[2] = Student(103, "Chirag")	[1], [2] filled
4	loop i = 0, 1, 2 calls display()	three lines printed

```
Roll: 101, Name: Aarav
Roll: 102, Name: Bhavna
Roll: 103, Name: Chirag
```

Question 5: Friend Function Declaration & Scope Resolution Flaws

Topic: Friendly Functions, Forward Declarations, Member vs. Non-Member

Step 1: Analyse the problem statement

- **Expected:** one non-member function that compares an `Engine`'s temperature with a `TelemetryAuditor`'s threshold, using friend access to both.
- **Actual:** three independent mistakes produce a cascade of errors.

Step 2: Errors found

Line	Classification	Defect
Line 4 / friend declaration in Engine	Compilation Error (undeclared type)	Only `class Engine;` is forward declared. GCC: "'TelemetryAuditor' has not been declared" at the friend declaration.
Line 29: `e.temperature > alertThreshold`	Compilation Error (scope)	GCC: "'alertThreshold' was not declared in this scope" (and "'int Engine::temperature' is private" until the friend declaration compiles).
Line 41: `v8.inspectDiagnostics(v8, audit)`	Compilation Error (misuse)	GCC: "'class Engine' has no member named 'inspectDiagnostics'".

Step 3: Root cause explanation

- **Task 1.** A friend function is **not a member**: it has no implicit `this` pointer. Inside `inspectDiagnostics`, the bare name `alertThreshold` is looked up as an ordinary variable and is not found. Every member must be reached through an explicit object: `a.alertThreshold`. Friendship only waives the `private` access check; it does not add an implicit object.
- **Task 2.** The member-selection syntax `obj.f()` only names members of `Engine`. A friend is declared inside the class for access purposes, but it belongs to the enclosing (global) namespace. It is called like any free function: `inspectDiagnostics(v8, audit);`.
- **Task 3 (forward declaration).** `Engine` mentions `TelemetryAuditor` in its friend declaration before that class has been declared. The compiler reads top to bottom and must know that the name is a type. Because `TelemetryAuditor` in turn needs `Engine` and the two classes refer to each other, the first one to appear needs a forward declaration (`class TelemetryAuditor;`) of the one that comes later. Forward declaring `Engine` (as the student did) is useless because `Engine` is defined immediately afterwards.
- **Good practice.** Pass const references: passing by value copies both objects for no reason.

Step 4: Corrected code

```
#include <iostream>

class TelemetryAuditor;                       // forward declaration

class Engine {
private:
    int temperature;
    int rpm;

public:
    Engine(int t, int r) : temperature(t), rpm(r) {}
    friend void inspectDiagnostics(const Engine &e, const TelemetryAuditor &a);
};

class TelemetryAuditor {
private:
    int alertThreshold;

public:
    TelemetryAuditor(int threshold) : alertThreshold(threshold) {}
    friend void inspectDiagnostics(const Engine &e, const TelemetryAuditor &a);
};

// Non-member friend: no 'this', so every member is reached through a named object
void inspectDiagnostics(const Engine &e, const TelemetryAuditor &a) {
    if (e.temperature > a.alertThreshold) {
        std::cout << "Warning: Engine overheat!" << std::endl;
    } else {
        std::cout << "Engine operating nominally at " << e.rpm << " RPM." << std::endl;
    }
}

int main() {
    Engine v8(105, 3200);
    TelemetryAuditor audit(100);

    inspectDiagnostics(v8, audit);            // ordinary function call
    return 0;
}
```

Step 5: Dry run and output

Step	Evaluation	Result
1	v8: temperature = 105, rpm = 3200; audit: alertThreshold = 100	objects built
2	inspectDiagnostics(v8, audit): e.temperature > a.alertThreshold: 105 > 100	true
3	if-branch executes	prints the overheat warning

```
Warning: Engine overheat!
```

SECTION B: PSEUDO-CODE WRITING QUESTIONS

Question 1: Corporate Vault Management with Static Data & Functions

Topic: Static Data Members, Static Member Functions, Encapsulation

Design overview

- **Instance state** (private, one copy per object): branchID, branchName, currentBalance.
- **Static state** (private, one copy shared by the whole class): totalVaultReserves and totalBranchesActive, defined and initialized outside the class to 0.0 and 0.
- Every instance method that changes a balance also updates totalVaultReserves, which keeps the aggregate consistent with the sum of branch balances.
- **STATIC METHODs** have no THIS: they may touch static attributes only, so they report class-wide numbers.
- Withdraw RETURNS BOOLEAN: it refuses (FALSE) when funds are insufficient or the amount is not positive.

Structured pseudo-code

```
CLASS BranchOffice
  PRIVATE:
    STRING  branchID
    STRING  branchName
    REAL    currentBalance
    STATIC REAL    totalVaultReserves
    STATIC INTEGER totalBranchesActive

  PUBLIC:
    CONSTRUCTOR(STRING id, STRING name, REAL openingDeposit)
      branchID       <- id
      branchName     <- name
      currentBalance <- openingDeposit
      totalVaultReserves  <- totalVaultReserves + openingDeposit
      totalBranchesActive <- totalBranchesActive + 1
    ENDCONSTRUCTOR

    METHOD Deposit(REAL amount)
      IF amount > 0 THEN
        currentBalance     <- currentBalance + amount
        totalVaultReserves <- totalVaultReserves + amount
      ENDIF
    ENDMETHOD

    METHOD Withdraw(REAL amount) RETURNS BOOLEAN
      IF amount > 0 AND amount <= currentBalance THEN
        currentBalance     <- currentBalance - amount
        totalVaultReserves <- totalVaultReserves - amount
        RETURN TRUE
      ELSE
        RETURN FALSE
      ENDIF
    ENDMETHOD

    STATIC METHOD GetVaultSummary()
      DISPLAY "Active branches : ", totalBranchesActive
      DISPLAY "Aggregate vault : ", totalVaultReserves
    ENDMETHOD

    STATIC METHOD GetGlobalLiquidityRatio(REAL targetReserve) RETURNS REAL
      IF targetReserve = 0 THEN
        RETURN 0.0
      ENDIF
      RETURN totalVaultReserves / targetReserve
    ENDMETHOD
ENDCLASS

// Static attributes are defined once, outside the class
```

```
REAL    BranchOffice::totalVaultReserves  <- 0.0
INTEGER BranchOffice::totalBranchesActive <- 0

BEGIN Main
  BOOLEAN ok
  REAL    ratio
  BranchOffice branch1("B001", "Downtown", 200000.0)
  BranchOffice branch2("B002", "Uptown",   300000.0)
  branch1.Deposit(150000.0)
  ok <- branch2.Withdraw(50000.0)
  BranchOffice::GetVaultSummary()
  ratio <- BranchOffice::GetGlobalLiquidityRatio(1000000.0)
  DISPLAY "Liquidity ratio: ", ratio
END Main
```

Trace table

The paper gives no opening deposits, so $200,000 (Branch1) and $300,000 (Branch2) are assumed for the trace.

Step	Operation	Branch1 balance	Branch2 balance	totalVaultReserves	totalBranchesActive
0	Initial state (static definitions)	-	-	0.00	0
1	Branch1 constructed (200,000)	200,000	-	200,000	1
2	Branch2 constructed (300,000)	200,000	300,000	500,000	2
3	Branch1.Deposit(150,000)	350,000	300,000	650,000	2
4	Branch2.Withdraw(50,000): 50,000 > 0 and <= 300,000, returns TRUE	350,000	250,000	**600,000**	2
5	GetVaultSummary()	-	-	prints 2 branches, 600,000	2
6	GetGlobalLiquidityRatio(1,000,000)	-	-	600,000 / 1,000,000	ratio = **0.6**

Question 2: Matrix Mathematical Engine with Arrays in Class & Friend Dot-Product

Topic: Arrays within a Class, Scope Resolution (::), Friend Functions

Design overview

- **Encapsulated array:** `grid[3][3]` is PRIVATE; clients use SetElement/GetElement, which validate indices (0 to 2) before touching memory.
- **Scope resolution:** the class body only declares the methods; definitions appear outside as `METHOD Matrix2D::Name()`.
- **Friend function:** ComputeCrossTrace is NOT a member, so it receives both matrices as parameters and reads `A.grid[i][i]` and `B.grid[i][i]` directly, which friendship permits.
- Matrices are passed by REF in the friend function so no 3 × 3 copy is made.

Structured pseudo-code

```
CLASS Matrix2D
  PRIVATE:
    ARRAY[3][3] OF REAL grid

  PUBLIC:
    METHOD Initialize()
    METHOD SetElement(INTEGER row, INTEGER col, REAL value)
    METHOD GetElement(INTEGER row, INTEGER col) RETURNS REAL
    METHOD Display()
    FRIEND FUNCTION ComputeCrossTrace(REF Matrix2D A, REF Matrix2D B) RETURNS REAL
ENDCLASS

// ---------- member functions defined outside the class ----------
METHOD Matrix2D::Initialize()
  INTEGER i, j
  FOR i <- 0 TO 2 STEP 1
    FOR j <- 0 TO 2 STEP 1
      grid[i][j] <- 0.0
    ENDFOR
  ENDFOR
ENDMETHOD

METHOD Matrix2D::SetElement(INTEGER row, INTEGER col, REAL value)
  IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
    grid[row][col] <- value
  ELSE
    DISPLAY "Error: index out of range"
  ENDIF
ENDMETHOD

METHOD Matrix2D::GetElement(INTEGER row, INTEGER col) RETURNS REAL
  IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
    RETURN grid[row][col]
  ENDIF
  DISPLAY "Error: index out of range"
  RETURN 0.0
ENDMETHOD

METHOD Matrix2D::Display()
  INTEGER i, j
  FOR i <- 0 TO 2 STEP 1
    FOR j <- 0 TO 2 STEP 1
      DISPLAY grid[i][j], " "
    ENDFOR
    DISPLAY NEWLINE
  ENDFOR
ENDMETHOD
```

```
// ---------- friend function (not a member: no THIS, no Matrix2D:: prefix) ----------
FUNCTION ComputeCrossTrace(REF Matrix2D A, REF Matrix2D B) RETURNS REAL
  REAL    sum
  INTEGER i
  sum <- 0.0
  FOR i <- 0 TO 2 STEP 1
    sum <- sum + (A.grid[i][i] * B.grid[i][i])      // private access granted by friendship
  ENDFOR
  RETURN sum
ENDFUNCTION

BEGIN Main
  Matrix2D m1, m2
  REAL result
  m1.Initialize()
  m2.Initialize()
  m1.SetElement(0, 0, 2.0)  m1.SetElement(1, 1, 4.0)  m1.SetElement(2, 2, 6.0)
  m2.SetElement(0, 0, 1.0)  m2.SetElement(1, 1, 3.0)  m2.SetElement(2, 2, 5.0)
  result <- ComputeCrossTrace(m1, m2)           // ordinary call, NOT m1.ComputeCrossTrace
  DISPLAY "Cross trace = ", result
END Main
```

Trace of the friend function: A_diag = [2, 4, 6], B_diag = [1, 3, 5]

Iteration	i	A.grid[i][i]	B.grid[i][i]	Product	sum after step
start	-	-	-	-	0.0
1	0	2	1	2 × 1 = 2	0 + 2 = 2
2	1	4	3	4 × 3 = 12	2 + 12 = 14
3	2	6	5	6 × 5 = 30	14 + 30 = 44
return	-	-	-	-	**44.0**

Question 3: Student Academic Roster via Array of Objects

Topic: Arrays of Objects, Member Functions, Sorting Objects

Design overview

- An ARRAY[3] OF StudentRecord is created by the default constructor, so the class must have one (blank values), then each element is filled with ReadData.
- CalculateResult is a PRIVATE helper, invoked automatically by ReadData, so a record can never hold marks without a matching percentage and grade.
- **Bubble sort (descending):** compare adjacent objects by GetPercentage(); swap whole objects when the left one is smaller; at most n - 1 passes.
- Sample data used: Asha (80, 90, 85), Ravi (70, 75, 80), Meena (90, 95, 92).

Structured pseudo-code

```
CLASS StudentRecord
  PRIVATE:
    INTEGER rollNo
    STRING  studentName
    ARRAY[3] OF REAL marks
    REAL      percentage
    CHARACTER grade
    METHOD CalculateResult()
      REAL total
      INTEGER k
      total <- 0.0
      FOR k <- 0 TO 2 STEP 1
        total <- total + marks[k]
      ENDFOR
      percentage <- total / 3
      IF percentage >= 85 THEN
        grade <- 'A'
      ELSE IF percentage >= 70 THEN
        grade <- 'B'
      ELSE IF percentage >= 50 THEN
        grade <- 'C'
      ELSE
        grade <- 'F'
      ENDIF
    ENDMETHOD

  PUBLIC:
    CONSTRUCTOR()                              // default constructor
      rollNo <- 0
      studentName <- ""
      marks[0] <- 0.0 ; marks[1] <- 0.0 ; marks[2] <- 0.0
      percentage <- 0.0
      grade <- ' '
    ENDCONSTRUCTOR

    METHOD ReadData(INTEGER r, STRING name, REAL m1, REAL m2, REAL m3)
      rollNo <- r
      studentName <- name
      marks[0] <- m1 ; marks[1] <- m2 ; marks[2] <- m3
      CalculateResult()
    ENDMETHOD

    METHOD GetPercentage() RETURNS REAL
      RETURN percentage
    ENDMETHOD

    METHOD Display()
      DISPLAY rollNo, " ", studentName, " ", percentage, "% Grade ", grade
```

```
    ENDMETHOD
ENDCLASS

BEGIN Main
  ARRAY[3] OF StudentRecord students          // 3 default-constructed objects
  StudentRecord temp
  INTEGER pass, j
  students[0].ReadData(1, "Asha",  80, 90, 85)
  students[1].ReadData(2, "Ravi",  70, 75, 80)
  students[2].ReadData(3, "Meena", 90, 95, 92)

  FOR pass <- 1 TO 2 STEP 1                   // n - 1 passes
    FOR j <- 0 TO 2 - pass STEP 1
      IF students[j].GetPercentage() < students[j+1].GetPercentage() THEN
        temp          <- students[j]
        students[j]   <- students[j+1]
        students[j+1] <- temp
      ENDIF
    ENDFOR
  ENDFOR

  DISPLAY "Rank list:"
  FOR j <- 0 TO 2 STEP 1
    students[j].Display()
  ENDFOR
END Main
```

Dry-run trace: percentages and grades

Student	Marks	Sum / 3	Percentage	Grade
Asha	80, 90, 85	255 / 3	85.00	A (85 >= 85)
Ravi	70, 75, 80	225 / 3	75.00	B
Meena	90, 95, 92	277 / 3	92.33	A

Dry-run trace: sorting passes and object swaps (descending)

Pass	j	Compare	Swap?	Array after step
start	-	-	-	[Asha 85.00, Ravi 75.00, Meena 92.33]
1	0	85.00 < 75.00 ? No	no	[Asha, Ravi, Meena]
1	1	75.00 < 92.33 ? Yes	swap objects 1 and 2	[Asha, Meena, Ravi]
2	0	85.00 < 92.33 ? Yes	swap objects 0 and 1	[Meena, Asha, Ravi]
end	-	(pass 2 inner loop runs only j = 0)	-	**[Meena 92.33 A, Asha 85.00 A, Ravi 75.00 B]**

Question 4: Flexible Logger with Overloaded Functions & Inline Helpers

Topic: Function Overloading, Default Arguments, Inline Functions

Design overview

- **INLINE FUNCTION** FormatPrefix: tiny, called on every log line, so the compiler is allowed to replace the call by its body and avoid call overhead.
- **Overload resolution is by argument count and types:** (STRING), (STRING, INTEGER, [STRING]), (INTEGER, STRING).
- **Important design catch.** If Overload 2 is declared with defaults for *both* severityLevel and timestamp, then `LogEvent("msg")` matches Overload 1 and Overload 2 exactly, and the call is **ambiguous** (confirmed: GCC "call of overloaded 'LogEvent(const char [2])' is ambiguous"; the same trap as Section A, Question 3).
- **Resolution used here:** the default is kept for timestamp, and severityLevel is a required parameter of Overload 2. A one-argument call therefore reaches Overload 1 only, and Overload 1 supplies the INFO level that a default severity would have provided.
- Severity text: 1 = LOW, 2 = MEDIUM, 3 = CRITICAL; anything else is UNKNOWN. Error codes are printed in hexadecimal.

Structured pseudo-code

```
INLINE FUNCTION FormatPrefix(STRING prefixTag) RETURNS STRING
  RETURN "[" + prefixTag + "] "
ENDFUNCTION

// Overload 1: general informational message (default prefix INFO)
FUNCTION LogEvent(STRING message)
  DISPLAY FormatPrefix("INFO") + message
ENDFUNCTION

// Overload 2: severity + timestamp (timestamp has a default value)
FUNCTION LogEvent(STRING message, INTEGER severityLevel, STRING timestamp = "00:00:00")
  STRING severityText
  IF severityLevel = 1 THEN
    severityText <- "LOW"
  ELSE IF severityLevel = 2 THEN
    severityText <- "MEDIUM"
  ELSE IF severityLevel = 3 THEN
    severityText <- "CRITICAL"
  ELSE
    severityText <- "UNKNOWN"
  ENDIF
  DISPLAY FormatPrefix(severityText) + "@ " + timestamp + " : " + message
ENDFUNCTION

// Overload 3: hardware fault diagnostic
FUNCTION LogEvent(INTEGER errorCode, STRING systemModule)
  DISPLAY "[HARDWARE_FAULT] Module: " + systemModule + " -> Error Code: 0x" + HEX(errorCode)
ENDFUNCTION

BEGIN Main
  LogEvent("System boot")                        // 1 argument
  LogEvent("Sensor drift", 2)                    // 2 arguments (STRING, INTEGER)
  LogEvent("Core temp high", 3, "14:32:07")      // 3 arguments
  LogEvent(255, "MOTOR_CTRL")                    // 2 arguments (INTEGER, STRING)
END Main
```

Trace: how each call is resolved and how defaults are filled

Client call	Arguments supplied	Overload chosen and why	Default filled in	Output
`LogEvent("System boot")`	1: STRING	Overload 1 (the only one callable with a single argument)	none needed (prefix INFO is fixed in the body)	[INFO] System boot
`LogEvent("Sensor drift", 2)`	2: STRING, INTEGER	Overload 2 (Overload 3 needs INTEGER first and a STRING cannot convert to INTEGER)	timestamp = "00:00:00"	[MEDIUM] @ 00:00:00 : Sensor drift
`LogEvent("Core temp high", 3, "14:32:07")`	3: STRING, INTEGER, STRING	Overload 2 (the only 3-parameter function)	none: all supplied	[CRITICAL] @ 14:32:07 : Core temp high
`LogEvent(255, "MOTOR_CTRL")`	2: INTEGER, STRING	Overload 3 (exact match; Overload 2 would need INTEGER to STRING, which does not exist)	none	[HARDWARE_FAULT] Module: MOTOR_CTRL -> Error Code: 0xFF

A C++ version of this design was compiled and run; it produced exactly the four output lines above.

Question 5: Hardware Register Bank with Return-by-Reference

Topic: Return by Reference, Arrays within a Class, Lvalue Semantics

Design overview

- **GetRegister** returns `REF INTEGER`: an alias to an element of the private array, so callers can read and overwrite a register without a separate setter.
- **Bounds safety:** for an invalid index the function must still return *something*, so it returns a reference to the dedicated `errorSink` variable. A bad write is absorbed there and can never corrupt memory outside the array.
- Both the registers and errorSink are members of the object, so they outlive the call: returning references to them is safe (contrast with a local variable).

Structured pseudo-code

```
CLASS RegisterBank
  PRIVATE:
    ARRAY[8] OF INTEGER registers
    INTEGER errorSink

  PUBLIC:
    CONSTRUCTOR()
      INTEGER i
      FOR i <- 0 TO 7 STEP 1
        registers[i] <- 0
      ENDFOR
      errorSink <- -1
    ENDCONSTRUCTOR

    METHOD GetRegister(INTEGER index) RETURNS REF INTEGER
      IF index >= 0 AND index < 8 THEN
        RETURN REF registers[index]          // alias of the real element
      ELSE
        DISPLAY "ALERT: register index ", index, " out of range"
        RETURN REF errorSink                 // harmless landing spot
      ENDIF
    ENDMETHOD

    METHOD DisplayState()
      INTEGER i
      FOR i <- 0 TO 7 STEP 1
        DISPLAY "R", i, " = 0x", HEX(registers[i])
      ENDFOR
    ENDMETHOD
ENDCLASS

BEGIN Main
  RegisterBank bank
  INTEGER readVal
  bank.GetRegister(0)  <- 170           // lvalue assignment: registers[0] = 170
  bank.GetRegister(1)  <- 255
  readVal <- bank.GetRegister(0)        // read through the reference
  bank.GetRegister(10) <- 999           // out of bounds: lands in errorSink
  bank.DisplayState()
  DISPLAY "readVal = ", readVal
END Main
```

Trace table

Step	Statement	Reference returned	registers[0..7]	errorSink
0	bank constructed	-	0 0 0 0 0 0 0 0	-1
1	GetRegister(0) <- 170	alias of registers[0]	170 0 0 0 0 0 0 0	-1
2	GetRegister(1) <- 255	alias of registers[1]	170 255 0 0 0 0 0 0	-1
3	readVal <- GetRegister(0)	alias of registers[0] (read)	unchanged	-1 (readVal = 170)
4	GetRegister(10) <- 999: 10 is not in 0..7, alert printed	alias of errorSink	unchanged	**999**
5	DisplayState()	-	R0 = 0xAA, R1 = 0xFF, R2..R7 = 0x0	999 (0x3E7)

How a function call can appear on the left of an assignment (Task 2)

- **Value categories.** A call to a function that returns an ordinary value (`INTEGER`) yields a temporary (an rvalue): it has no storage the program can name, so `f() = 5` is meaningless.

- A call to a function that returns a reference (`REF INTEGER`, `int&` in C++) yields an **lvalue**: the expression designates the existing object that the reference is bound to.
- Therefore `bank.GetRegister(0) = 170` is evaluated as: call the function, receive an alias of `registers[0]` (internally the object's address), then perform the ordinary assignment to that object. The assignment changes the object inside the bank, not a copy.
- **Condition:** the referred-to object must outlive the call (member, caller-owned data, heap or static object). Returning a reference to a local variable gives a dangling reference (Section A, Question 1).